

10

Principal Components Analysis

CHAPTER OBJECTIVES
■ Learn what principal components analysis (PCA) is and why it can be useful as a multi-variate methodology. ■ Understand how PCA can be obtained from the basic eigenvalue–eigenvector problem. ■ Appreciate the differences between PCA and exploratory factor analysis (EFA), and why they are not the same procedure and should not be treated as such. ■ How to potentially make substantive sense out of components derived from PCA and when to recognize when a PCA may not have resulted in anything useful. ■ How to conduct a basic PCA in Python and how to interpret results.

You will recall from our discussion of **discriminant analysis** that one of the objectives of the method, in addition to correctly classifying observations, was to reduce the **dimensionality** of the data. That is, the discriminant functions we obtained were derived from reducing the dimensionality of the original data set into one or more functions that could, essentially, “take their place” when trying to classify the data. In other words, instead of using several observed variables on a given number of dimensions, perhaps we could reduce the dimensionality into **linear composites** and then use those linear composites to discriminate between groups. The idea of dimension reduction was thus very important to discriminant analysis, and, as we will see, plays a central role in the methodology considered in the current chapter, that of principal components analysis (PCA).

10.1 What Is Principal Components Analysis?

Principal components analysis (PCA) is a **dimension-reduction** technique (Jolliffe, 2002). Sometimes it is also referred to as a **data-reduction** technique. Both designations are appropriate so long as one understands what each means. It is essential to learn what PCA can do just as much as it is important to understand what it cannot do

so that you are not using the procedure with a sense of hoping it will tell you something about your data and scientific hypotheses that it cannot provide you with. As emphasized throughout this book, understanding the **limitations** of a statistical procedure is just as, if not more important, as understanding what it can do. Hence, it is imperative that we survey PCA in a bit of detail here to understand its underlying mechanics, and, in a strong sense, better appreciate just how limited it can be in addressing scientific problems. An example will help in motivating its purpose.

Suppose a researcher has 1,000 variables and has these variables at his or her disposal. Each of these variables can be said to represent a **single dimension**. For now, the exact nature of these variables or what they are substantively is unimportant. We will simply assume for now that they are measurable on a **continuous** scale. Now, it stands that these 1,000 variables, as a group, account for a certain amount of variance. That is, the first variable, second variable, etc., all have variances associated with them, and hence when we consider the set of variables in entirety, the total number of variables accounts for a well-defined amount of variance; that is, a **totality** of variance. Hence, if a researcher were asked how much variance the set of variables accounts for, he or she would sum up the individual variances of each variable and obtain a sum total variance.

As a simple example, one that we will elaborate on in this chapter, consider the following two variables x and y , which we build in Python:

```
x = [.00, .90, 1.80, 2.60, 3.30, 4.40, 5.20, 6.10, 6.50, 7.40]
y = [5.90, 5.40, 4.40, 4.60, 3.50, 3.70, 2.80, 2.80, 2.40, 1.50]
pca_data = pd.DataFrame(x, y)
pca_data

Out[84]:
5.9  0.0
5.4  0.9
4.4  1.8
4.6  2.6
3.5  3.3
3.7  4.4
2.8  5.2
2.8  6.1
2.4  6.5
1.5  7.4
```

What is the variance of variable x ? We can easily compute this in Python using the **statistics** package:

```
import statistics
statistics.variance(x)

Out[74]: 6.266222222222223
```

We see that the variance of x is equal to 6.27. Likewise, the variance of y is computable:

```
statistics.variance(y)
Out[75]: 1.9133333333333338
```

The **total variance** accounted for by variables **x** and **y** is therefore equal to the sum of the variances:

```
total_variance = statistics.variance(x) + statistics.variance(y)
total_variance
```

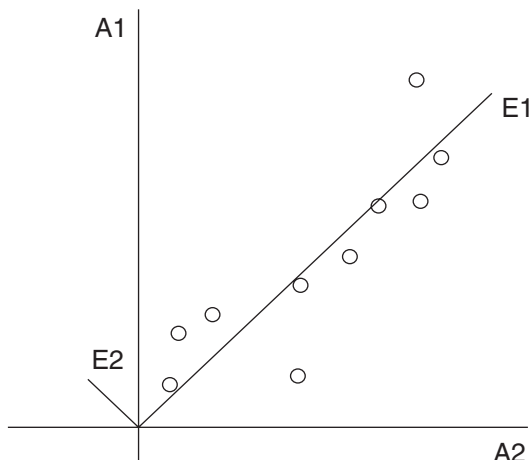
```
Out [77]: 8.179555555555556
```

We therefore see that the total variance accounted for by both variables is equal to 8.18 (rounded up). The question asked by PCA is a remarkably simple one, yet historically quite profound. It is the following:

Can the total variance in the two variables be re-expressed by a transformation?

This question is the only question answerable by PCA, so it is imperative that we examine what this question actually means and how it is answered by PCA. What does it mean to “re-express” variance and why would we want to do such a thing in the first place? The first step to understanding this is to appreciate that **PCA will not fundamentally alter or change the information present in your data**. It will simply **transform the information** on variables onto new axes and dimensions, similar in this respect to what occurred in discriminant analysis. Recall in discriminant analysis that the fact we obtained a number of discriminant functions from our analysis did not “change” the data. It simply **reorganized** it. Analogously, this is likewise what occurs in PCA as well, though the constraints imposed on the procedure will be different. Though PCA operates differently, both techniques seek to **reduce dimensionality**. Much of multivariate (and univariate, for that matter) analysis is about **dimension reduction**. Even ANOVA, “simple” univariate ANOVA, features dimension reduction, though it may not at first glance be apparent.

Consider the following diagram as a graphical depiction of what occurs in PCA, where in this case we only have two original variables:



In the plot, A1 and A2 are the **original axes** of the variables. One interpretation of PCA is that it **rotates the original axes** to E1 and E2, where E1 is the first principal component and E2 is the second. Notice that E1 fits the data very well, while E2 does not. By “fits the data” in this sense, we no longer mean it minimizes the **vertical**

distances from data points to the line as in traditional least-squares. Rather, PCA seeks to minimize the **squared perpendicular distances** instead. Notice as well that the newly derived components are **perpendicular** to each other. That is, they are **orthogonal** to one another at 90-degree angles. This is one vital characteristic of PCA – it generates dimensions that are orthogonal to one another and hence each dimension presumably accounts for an individual proportion of variance in the original variables.

10.2 Principal Components as Eigen Decomposition

Though there are different ways of obtaining components in PCA, by far the most common way is to execute an **eigen decomposition** on the **covariance** or **correlation** matrix. Another way is **singular-value decomposition** (SVD). SVD can be used on general rectangular matrices, whereas eigen decomposition is restricted to square matrices (Mair, 2018). In this sense, PCA can be considered nothing more than obtaining the eigenvalues and eigenvectors of a covariance or correlation matrix. Though we have briefly surveyed this process of eigen decomposition earlier in the book, we briefly review it here as it is fundamental to PCA. It is imperative that you see how PCA can be likened to nothing more than an eigen decomposition, and hence, in this respect, is a mathematical **abstraction** first and foremost. What we mean by this is that PCA is an operation on mathematical objects and will not **necessarily** correspond to anything meaningful in **your** particular data.

For a square matrix **A**, it is well known that a scalar λ and vector **x** can be found such that the following equality is true:

$$\mathbf{Ax} = \lambda \mathbf{x}$$

What this means in English is that for a given matrix **A** multiplied by a vector **x**, this product represents the same quantity as taking that vector and multiplying it by a unique scalar λ . Now, that does not help much does it? Here is a better way of looking at it. The vector **x** is being **transformed** by the matrix **A**. This transformation is equivalent in nature to transforming **x** by simply multiplying it by that unique scalar λ . The historical significance of the above equality cannot be overstated. It literally serves as the foundation for many statistical techniques, especially those in multivariate analysis. It is present in univariate techniques such as ANOVA just as it is in more complex multivariate techniques, though it is only in multivariate tools where it is most apparent. The “special vector” is called an **eigenvector**, and the “special scalar” is called an **eigenvalue**. There will typically be as many eigenvectors extracted from **A** as there are rows and columns, and each eigenvector will be associated with its own eigenvalue. Eigenvectors, however, are only unique up to multiplication by a scalar. The eigenvectors, as we will see, are the coefficients to what will become the principal components, and the eigenvalue will be the variance explained by each component.

We can extract eigenvalues and eigenvectors in Python quite easily. First, we will construct the covariance matrix of our two variables, *x* and *y*:

```
data = np.array([x, y])
data
```

```

Out[85]:
array([[0.,  0.9, 1.8, 2.6, 3.3, 4.4, 5.2, 6.1, 6.5, 7.4],
       [5.9, 5.4, 4.4, 4.6, 3.5, 3.7, 2.8, 2.8, 2.4, 1.5]])

covMatrix = np.cov(data, bias = False)
covMatrix

Out[88]:
array([[ 6.2662222, -3.3811111],
       [-3.3811111,  1.9133333]])

```

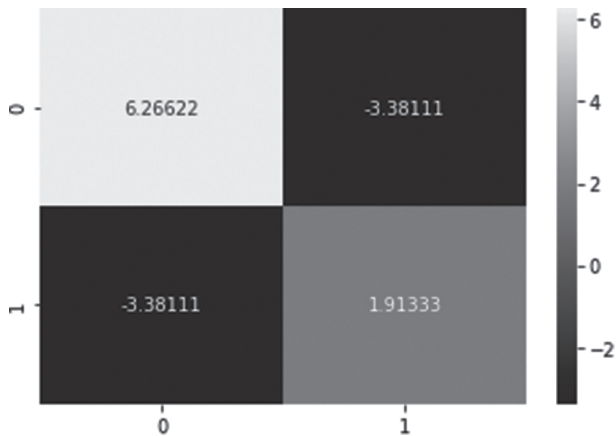
We see from the above that the variance of the first variable is 6.26 while the variance of the second variable is 1.91. The covariance between variables is -3.38 and appears in the off-diagonal of the matrix. We can also obtain a **heatmap** of the covariance matrix using **seaborn**:

```

import seaborn as sn
sn.heatmap(covMatrix, annot=True, fmt='g')

Out[90]: <matplotlib.axes._subplots.AxesSubplot at 0x75b6438>

```



Now that we have successfully constructed the covariance matrix, it is a simple matter to extract eigenvalues and eigenvectors from it. For this, we can use numpy's **linalg** function, which stands for "linear algebra" since eigenvalues and eigenvectors are mathematical objects of linear algebra and are a central topic of that area of study:

```

import numpy.linalg as la
eigen = la.eig(covMatrix)

eigen

Out[94]:
(array([8.11082525, 0.06873031]), array([[ 0.87785621,
 0.47892429],
      [-0.47892429,  0.87785621]]))

```

We see from the above that the two eigenvalues are equal to 8.11 and 0.06, with the first eigenvector equal to 0.878 and -0.478 , while the second eigenvector having

elements 0.479 and 0.878. What we have done with this simple eigen extraction can in fact be considered a rudimentary PCA on a covariance matrix. “Extraction” is probably not the best word here, since, as we will see in the next chapter, this could easily be confused with the process inherent in exploratory factor analysis (EFA) where a user typically designates in advance the number of factors to extract. In PCA, whether we keep a subset of the available components or not, they are still all computed when we perform the procedure. Had we wanted to be a bit more specific about the values we were “extracting,” we could have requested and called them as follows:

```
eigenvalue, eigenvector = la.eig(covMatrix)
eigenvalue, eigenvector

Out[96]:
(array([8.11082525, 0.06873031]), array([[ 0.87785621,
0.47892429],
      [-0.47892429,  0.87785621]]))
```

This better indicates that we are extracting first the eigenvalues followed by the corresponding eigenvectors.

10.3 PCA on Correlation Matrix

Above we conducted a basic PCA on a covariance matrix. Had we wished to perform it on a **standardized covariance matrix**, also known as a **correlation matrix**, we would have computed:

```
import pandas as pd
data = {'x': [0, 0.9, 1.8, 2.6, 3.3, 4.4, 5.2, 6.1, 6.5, 7.4],
'y': [5.9, 5.4, 4.4, 4.6, 3.5, 3.7, 2.8, 2.8, 2.4, 1.5]}
df = pd.DataFrame(data, columns = ['x', 'y'])
corrMatrix = df.corr()
print(corrMatrix)

           x          y
x  1.000000 -0.976475
y -0.976475  1.000000
```

Why is the correlation perfectly **symmetric**? Notice that the off-diagonals of the correlation matrix are exactly the same. This is because what is contained in the main diagonal, from top left to bottom right, are correlations of the variables with themselves, and hence these correlations will always equal 1.0. In the off-diagonal are the actual correlations between the two variables and just as the covariances were the same in the covariance matrix, these correlations will also be identical. We now compute an eigen decomposition on the correlation matrix:

```
import numpy.linalg as la
eigen = la.eig(corrMatrix)
eigen
```

```
Out[83]:
(array([1.97647522, 0.02352478]), array([[ 0.70710678,
 0.70710678],
      [-0.70710678,  0.70710678]]))
```

We see that the eigenvalues extracted are equal to 1.976 and 0.023 respectively for the two components. The first eigenvector is that of 0.707 and -0.707, while the second is 0.707 and 0.707. Notice that **the eigenvalues and eigenvectors extracted from the correlation matrix are not the same as those extracted from the covariance matrix**. Eigen analysis on correlation matrices will usually be different than on covariance matrices. We will revisit this important point later in the chapter. Which matrix you subject to the PCA can have important consequences.

10.4 Why Icebergs Are Not Good Analogies for PCA

The mathematics of PCA are what they are, and as we will see, they are relatively simple and straightforward (at least at the level we are surveying PCA). As we have seen, **PCA is nothing more than a transformation of information onto new dimensions**. That transformation is a technical one, a mathematically rigorous way of accounting for the same amount of variance in variables as by the original variables subjected to the analysis. But what does this really mean on a conceptual level? Not necessarily that much. Though it may be tempting to associate PCA (and its cousin, factor analysis) with “uncovering latent dimensions that underlie observed variables giving rise to them,” as with any statistical method, we must be very careful not to project our own “substantive dreams” on the “innocent” mathematics. The mathematics are first and foremost **abstract**, though they may have historically been motivated (and often are) by substantive physical problems. **The point is, however, that the PCA does not know what you are thinking. It is only doing what it does, and that is a mathematical abstract transformation.** That is the only thing we can be sure of, for certain. All PCA does is perform a transformation that generates **orthogonal linear combinations of the original variables**. That’s it! It does not do anything more that the scientist may have in mind. Does that mean it may not “mean” more in a particular substantive context? Absolutely not. But the point is that, by itself, PCA is quite “boring.” It does not necessarily mean you are “uncovering” (in a physical sense) anything. Hence, when a scientist or researcher uses PCA with the goal of “uncovering” dimensionality, it is a matter of philosophical truth that PCA may not be able to do this for you in the way you may be thinking.

One way to think about PCA, and a lot of mathematics for that matter, is to consider the transformation as simply a **parallel transfer** of information. What we mean by “parallel” here is to suggest that in the transformation, **no new knowledge is *necessarily* being created; it is simply being moved from one form to another**. Though it may hint or suggest new knowledge, it is not a necessary implication of performing a PCA. A parallel transfer, in the way we are using the term, is simply “re-organizing” information from one format into another, such as illustrated in Figure 10.1.

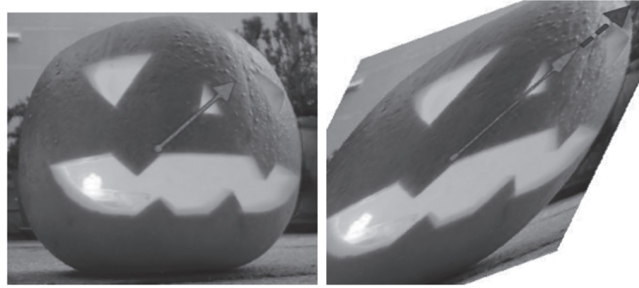
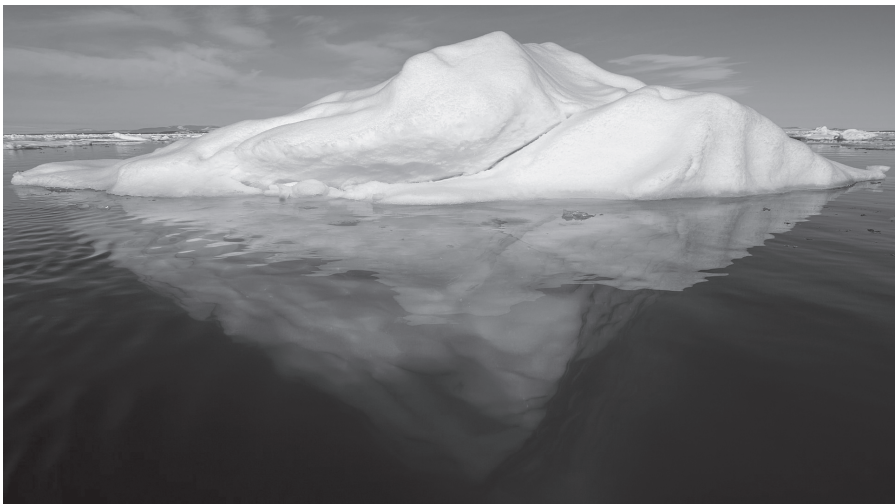


Figure 10.1 Pumpkin with an eigenvector visible before the transformation (left). Pumpkin after the transformation with the eigenvector elongated by an eigenvalue (shown by the dotted arrow) associated with the eigenvector (right). The essential information in the pumpkin remains, but has been subjected to a linear transformation. Image used with permission.

In Figure 10.1, the right image features a pumpkin undergoing a transformation. The original shape is on the left side. The principal component vector, figuratively at least, is also drawn in. Notice that after the transformation, the vector is elongated (indicated by the dashed arrow), since it accounts for more variance in the transformed object than in the original. Though this analogy is not perfect, it is nonetheless useful for grasping what PCA accomplishes, which is nothing more than a **transformation of information onto new axes**, such as in the case of the pumpkin, or, Euclidean space more generally. The object of the pumpkin need not exist for the transformation to make sense. **PCA can be entirely an abstract operation on an abstract space with no recourse to physical representation.** Hence, when you perform a PCA on your variables, it does not necessarily mean the abstract system you are imposing on your physical objects will necessarily make the mathematical transformation “meaningful.” It **may** make it meaningful, but it need not **necessarily**. This is very important to understand.

Despite our caveat above, some folks still like to interpret PCA using an iceberg analogy, where above the water are the observed variables and below it are the underlying components, as illustrated in the following:



This iceberg image is not an ideal representation of the PCA idea, however, since there is no reason why the iceberg below the surface of the water should have more information in it than the portion of the iceberg above water. The spirit of the analogy is that PCA will reveal the underlying components to the variables, but the iceberg analogy encourages us to see these underlying components as “hidden” and larger than the ice above the water. If you make the amount of information above and below the water line equal, then the analogy stands more of a chance at being a fair one. But even then, does the analogy hold? That is, why would the transformed information contain more “essence” (even if not more physical space) than the original information? The truth is that there is not necessarily any reason why this should be the case, because, as mentioned, it is simply a transformation of information onto new axes. However, in some applications of PCA, it may make sense to conceptualize it in this manner, but this is **only when the mathematics are merged with substantive theory and it makes sense to do so**. Otherwise, there is simply no basis to conclude on the mathematics alone that PCA or any other technique in statistics “unearths” underlying dimensions. It simply performs a rather anticlimactic and “boring” (I’m being flip, mathematics is far from boring) linear transformation. Anything more than that must come from substantive theory on which you, as a scientist, carry the burden of justifying what it might mean beyond that. The “merger” of statistics with science necessitates this point being very clear – **an analytical method or system cannot impose qualities onto a physical system**. It can only operate on an existing system if there is one to begin with. Hence, whether an underlying dimensionality has been “discovered” or “unearthed” will depend fully on whether you have the substantive material in your science (not in your statistics) to make that argument. Bottom line? A PCA may be nothing, or it may be something, depending on the quality of your data and the scientific inferences you can reasonably make. Otherwise, PCA is simply a mathematical abstraction and every scientist who uses the tool (or any statistical tool) needs to be aware of this.

10.5 PCA in Python

Though one can perform a basic PCA as we have done above using raw linear algebra and computing eigenvalues and eigenvectors, much more common of course in Python is to use a pre-programmed routine or function. We now demonstrate how we can obtain the same PCA as above, but through using `pca.fit()`. For this example, we will build our data file using an `array()` statement:

```
A = np.array([[5.9, 0.0], [5.4, 0.9], [4.4, 1.8], [4.6, 2.6], [3.5, 3.3], [3.7, 4.4], [2.8, 5.2], [2.8, 6.1], [2.4, 6.5], [1.5, 7.4]])
```

A

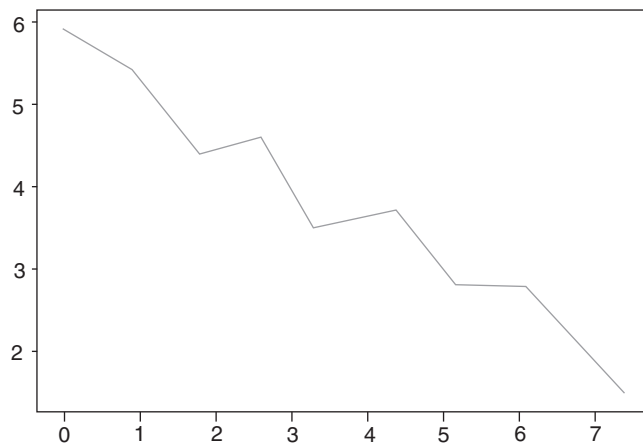
Out[97]:

```
array([[5.9, 0. ],
       [5.4, 0.9],
       [4.4, 1.8],
       [4.6, 2.6],
       [3.5, 3.3],
```

```
[3.7, 4.4],
[2.8, 5.2],
[2.8, 6.1],
[2.4, 6.5],
[1.5, 7.4]])
```

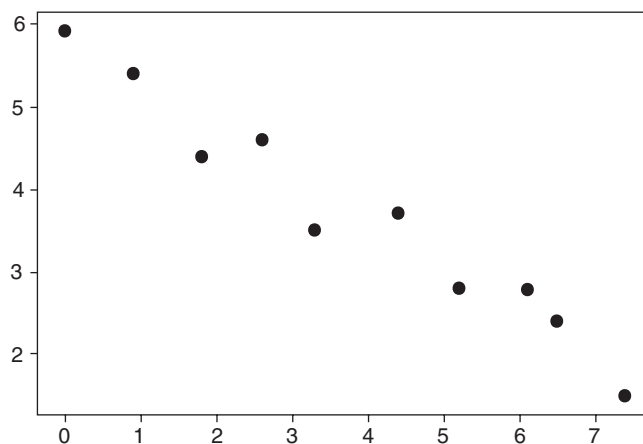
Let us obtain a plot of the data:

```
import matplotlib.pyplot as plt
plt.plot(x, y)
```



This does not tell us of the actual data points, so we may prefer the following option, where we can see the actual individual data more clearly:

```
plt.plot(x, y, 'o', color='black')
Out[107]: [<matplotlib.lines.Line2D at 0x1e66ce80>]
```



Recall that when we extracted eigenvalues and eigenvectors from our data earlier, there were two sets. This is because the problem had two variables. When we request a PCA in

Python, we can specify the number of components to “extract.” We put “extract” in quotes here again to emphasize that in PCA, regardless of the number of components you specify to extract, the procedure is, under normal circumstances, always extracting as many components as there are variables. Hence, when we request the number of components to be less than the number of variables, all we are asking the software to do is to report fewer of the components. When we conduct factor analysis in the following chapter, we will see this is not the case. In factor analysis, **the number of factors that you choose to extract or retain will usually have a mathematical consequence on the structure of those and remaining factors.** This is a very important distinction.

With the above caveat in mind, we conduct a PCA in which two components are extracted, which is the maximum number for this analysis since there are only two variables:

```
from sklearn.decomposition import PCA
pca = PCA(2)

pca.fit(A)
Out[100]:
PCA(copy=True, iterated_power='auto', n_components=2, random_
state=None,
    svd_solver='auto', tol=0.0, whiten=False)

print(pca.components_)
[[ 0.47892429 -0.87785621]
 [-0.87785621 -0.47892429]]
```

Above are the eigenvectors associated with the components analysis. Notice that we have conducted the components analysis here directly on the data. That is, contrary to what we did earlier in the chapter, we did not need to first construct a covariance matrix. We see as well that the signs of the coefficients are a bit different from the original extraction of eigenvalues and eigenvectors. This is common across different software programs and is not a concern. We can obtain the eigenvalues for the PCA through `pca.explained_variance_`:

```
print(pca.explained_variance_)
[8.11082525 0.06873031]
```

As we saw earlier, the first eigenvalue, that of 8.11, is dominant among the two, suggesting that the first component explains about as much of the variance in the variables as both original variables. We can compute the proportion of variance explained by summing the variance and dividing each variance by that sum. That is, since the total sum is equal to 8.18, the proportion of variance accounted for by the two components is:

Component 1 Proportion of Variance Explained = $8.11 / 8.18 = 0.991$

Component 2 Proportion of Variance Explained = $0.06 / 8.18 = 0.007$

To emphasize our previous point, suppose we had, instead of requesting two components, requested only a single component:

```
from sklearn.decomposition import PCA
pca = PCA(1)
pca.fit(A)
```

```

Out[574]:
PCA(copy=True, iterated_power='auto', n_components=1, random_
state=None,
      svd_solver='auto', tol=0.0, whiten=False)

print(pca.components_)
[[ 0.47892429 -0.87785621]]

print(pca.explained_variance_)
[8.11082525]

```

Notice that the request for a single component rather than two does not change the structure of the first component in terms of its eigenvector; nor does it alter the proportion of variance explained. Again, we demonstrate this to contrast it with exploratory factor analysis of the following chapter, where the number of factors requested in the extraction will typically (for most versions of factor analysis) make a difference to both the vector structure of it, as well as the proportion of variance accounted for by that factor.

10.6 Loadings in PCA: Making Substantive Sense Out of an Abstract Mathematical Entity

We have seen how we can extract eigenvalues and eigenvectors from a covariance or correlation matrix. But what are these things? It needs to be emphasized at the outset that eigenvalues and eigenvectors are mathematical “realities.” That is, their existence can be said to be purely abstract in that they naturally fall out of mathematical derivation. They characterize essential information inherent in a linear transformation. Now, to understand what this means requires a bit of maturity into understanding what a mathematical result actually is and why it may not be what you have already preconceived. Most people’s perceptions of mathematics is far from the reality of what mathematics actually is. A very simple, if not trivial example will help in appreciating what an abstract mathematical consequence is, one that does not require even understanding the nature of the objects on which the reasoning is taking place. On an abstract level, a pure mathematician may not care that much about the true entity of the objects with which he or she deals. What a mathematician is often most interested in is how they **relate** to one another.

Imagine we have three objects, A, B, and C. What these objects are is not important. What is important is knowing they can be subjected to operations such as addition and subtraction. But again, the “nature” of A, for instance, is immaterial for our purposes (so far). Now, consider the following statements:

If C is greater than A, and B is in between A and C, but not equal to either of them, it stands therefore that C is greater than B.

Wow! What we have just performed is **deductive logic** on relations between objects. Notice that the statement makes good sense even without knowing, understanding, or any **insight** into the objects on which the statements were made! We do not know

what A, B, or C are, yet we know the statement holds true based on our own powers of reasoning that tells us it makes good sense. Nowhere in our query did we have to evaluate the **nature** of A, B, or C. It was good enough to simply study how they relate to one another.

Now, what on earth does any of this have to do with PCA? Everything! The above exemplifies how eigenvalues and eigenvectors “exist” distinct from us giving them any substantive or scientific meaning. That is, they are mathematical objects, neutral, and are a consequence of mathematical theorems and derivation. You might say they have no substantive meaning until we give them one. This, in a strong sense, is what PCA does. PCA gives meaning to eigenvalues and eigenvectors such that they may be of use to the scientist in making sense out of his or her data. But, fundamentally, eigenvalues and eigenvectors are, first and foremost, abstract mathematical objects.

So, what do these objects represent then? Eigenvalues and eigenvectors represent different values depending on the given statistical method. However, in PCA, as we have seen, the eigenvalues represent **variances**, while the elements of the corresponding eigenvectors represent **loadings** on the given components. That is, the eigenvalues correspond to the **variance of the components**. The components are the newly derived entities on which the original variables have been translated. The sum of components computed from the analysis explains just as much variance as the original variables, but the hope is that by transforming the original information, the scientist can explain most of the original variance by reference to the components only; hence the idea of variance consolidation or “data reduction” we emphasized earlier. To summarize:

Eigenvalues in PCA = variances of components.

Eigenvectors in PCA = loadings of components.

But just how can we use the loadings? We can potentially use the loadings to identify or “name” the given component. Loadings with greater size suggest the given component is “composed” more of that variable than others, but there are many difficulties with this. Let us explore a couple of them.

10.7 Naming Components Using Loadings: A Few Issues

Though we can use loadings to help “identify” the nature of a component substantively, a few caveats and warnings are in order. If these are not heeded or appreciated, the user runs the risk of concluding something from the data that simply is not justified. We consider these caveats in some detail:

- Whatever component is derived is based on the variables inputted into the analysis, and no other variables. Hence, the selection of variables for the PCA is the most important and crucial step of the entire components analysis. Of course, it may seem obvious that the components analysis is a function of the variables subjected to the analysis, but it is a point that can easily be lost on users. How is such unawareness evident? It is by drawing conclusions of the “physical nature” type that simply may not be warranted or justified for the area under investigation. For example, if a social

scientist subjects 100 variables to a PCA, the PCA is not “uncovering” components or making a “discovery” of any sort. As we have said, all the PCA is accomplishing is a transformation. Hence, you guessed it, had more or less variables been subjected to the PCA than the original 100, the results and findings would likely change! That is, your PCA is only as good as the variables you subject to it. You should choose those variables very carefully, otherwise **your PCA may simply be misspecified**. As a scientist, spend 99% of your time selecting the correct candidate variables and 1% conducting the PCA! All loadings obtained in PCA are relative to all other variables subjected to the analysis.

- Related to the above point, social or psychological components may be more difficult to substantiate than components derived from an obvious physical process. For example, in digital images, components can be extracted that account for most of the variance in the image (Izenman, 2008). This is a guaranteed physical process, however. And though we can apply PCA to data collected on social data, it is more difficult to make the argument that such extracted components represent physical “building blocks” as they do in things like the digital image. Indeed, early motivations behind PCA were in representing physical and biological objects. Descriptions of such systems may be more “convincing” than claiming that IQ has a three-component structure, for instance. As a researcher, especially one doing social science, you should be aware of these distinctions. Do not simply subject your data to PCA and draw conclusions of the type a physical scientist would draw unless you have a very strong argument that your components are a “breakdown” of a physical or similar process as that for a physical object.
- Components derived from the correlation matrix will usually not be the same as those obtained from the covariance matrix. Hence, component extraction and retention can be quite subjective since they depend on the original matrix subjected to the analysis. This is related to the above point, since it tells us that “**existence**” of components underlying variables may be the wrong way to think of PCA. All PCA is doing (for the millionth time!) is a **transformation**, and so the details of that transformation must be clearly understood if one is to understand what has actually been obtained in the PCA. As we will see when we survey exploratory factor analysis in the following chapter, this concern is even of more significance in that procedure. The bottom line recommendation here is to not immerse yourself into issues of “existence” (or at least be very cautious about them) when performing either PCA or factor analysis, as they will not necessarily arise in the context you are working in. These procedures are not akin to an archeological dig where you eventually have a good sense of certainty in what has been found and can touch and feel it.
- PCA results may not be comparable across studies. It would be a mistake to conclude that you found three components while another study found four without unpacking **all of the details inherent in and a part of each study**. Even if assuming the same measures were used, they were surely used on different populations, and hence the details of each population should be advanced if any comparison is to be made. And as mentioned, even the inclusion of an additional variable can “throw off” your component structure, and hence comparing studies where different variables were used (even if that difference is slight) can have significant consequences to each analysis.



In using PCA (or any other statistical method for that matter), the user needs to be intimately familiar with what can vs. cannot be concluded from the procedure or algorithm. For example, PCA does not “discover” components. Rather, it merely transforms a high-dimensional “structure” onto another. Hence, an argument that a component “exists” is usually a weak one, unless one is working with physical variables (or equivalent substantively defensible variables) on which such a conclusion can be justified by reference back to the physical objects. Concluding that a component structure underlies something like intelligence, for instance, will usually require a bit more work and substantive evidence than in the purely physical (and simpler, in this manner of speaking) case.

10.8 Principal Components Analysis on USA Arrests Data

We now demonstrate a full components analysis on a more complex and realistic data set than the previous one on two variables. Our data contains statistics on the number of arrests per 100,000 residents in all states in the USA in the year 1973. The variables in our data are **murder**, **assault**, **urbanpop**, and **rape**, and the total number of observations is 50. The murder, assault, and rape variables are the number of such crimes per 100,000 cases, while urbanpop is simply the percent of the population that is urban. The data file is in `.csv` format and we use `pandas` to read the file, printing only the first 10 cases:

```
data = pd.read_csv('usarrests.csv')
data.head(10)
```

Out[583]:

	Unnamed: 0	Murder	Assault	UrbanPop	Rape
0	Alabama	13.2	236	58	21.2
1	Alaska	10.0	263	48	44.5
2	Arizona	8.1	294	80	31.0
3	Arkansas	8.8	190	50	19.5
4	California	9.0	276	91	40.6
5	Colorado	7.9	204	78	38.7
6	Connecticut	3.3	110	77	11.1
7	Delaware	5.9	238	72	15.8
8	Florida	15.4	335	80	31.9
9	Georgia	17.4	211	60	25.8

We select only the quantitative variables for analysis. That is, we will not include the state variable, as it is nominal in nature. Incidentally, when selecting variables from a dataframe, if you misspell the variable, it will likely come out as `NaN` as follows:

```
df = pd.DataFrame(data, columns=['Murder', 'Assault', 'Urbanpop',
                                'Rape'])
df
```

Out[351]:

	Murder	Assault	Urbanpop	Rape
0	13.2	236	NaN	21.2

```

1      10.0      263      NaN  44.5
2       8.1      294      NaN  31.0
3       8.8      190      NaN  19.5
4       9.0      276      NaN  40.6

```

Our error above was requesting `Urbanpop` instead of `UrbanPop` (i.e. with the capital P). When we request it correctly, we obtain the correct data layout:

```

df = pd.DataFrame(data, columns=['Murder', 'Assault', 'UrbanPop',
                                'Rape'])
df

```

```

Out[353]:
      Murder  Assault  UrbanPop  Rape
0      13.2     236      58    21.2
1      10.0     263      48    44.5
2       8.1     294      80    31.0
3       8.8     190      50    19.5
4       9.0     276      91    40.6
5       7.9     204      78    38.7

```

Now that our data file is set up correctly, let's obtain some means of the variables before we conduct the PCA. For this, we will use `np.mean()`:

```

import numpy as np
np.mean(df)

Out[355]:
Murder      7.788
Assault    170.760
UrbanPop    65.540
Rape       21.232
dtype: float64

```

The variances of the variables are given by `np.var()`:

```

np.var(df)

Out[356]:
Murder      18.591056
Assault    6806.262400
UrbanPop    205.328400
Rape       85.974576
dtype: float64

```

As we will see later when we also compute the variances using pandas, `np.var()` uses “ n ” in the denominator for the sample variance and not “ $n - 1$ ” as does pandas. That is, the computation of the variance above for murder of 18.591056 is computed as

$$S^2 = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n}$$

rather than by

$$s^2 = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n-1}$$

The difference between the two calculations will usually not be that great, especially for large sample size, but typically, since we usually desire an unbiased estimator, the second choice with $n-1$ is what is reported by most software. To get the unbiased estimate in numpy (i.e. with $n-1$), you can code `np.var(df, ddof=1)`, where `ddof=0` is the default with simply n in the denominator. Regardless of what denominator we use, we nonetheless notice that the variances are quite different from one another. Hence, if we run the components analysis on the raw data, the assault variable with its quite large variance might dominate the solution. The way around this is to **standardize** the data. We do so using `StandardScaler()`:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(df)

Out[357]: StandardScaler(copy=True, with_mean=True, with_std=True)

scaled_data = scaler.transform(df)
scaled_data

Out[358]:
array([[ 1.25517927,  0.79078716, -0.52619514, -0.00345116],
       [ 0.51301858,  1.11805959, -1.22406668,  2.50942392],
       [ 0.07236067,  1.49381682,  1.00912225,  1.05346626],
       [ 0.23470832,  0.23321191, -1.08449238, -0.18679398],
       [ 0.28109336,  1.2756352 ,  1.77678094,  2.08881393],
       [ 0.02597562,  0.40290872,  0.86954794,  1.88390137],
```

These data are now standardized, but we notice that Python has removed the headers of the variables. We can easily get the headers back onto our data file via the following:

```
scaled_data_df = pd.DataFrame(scaled_data, columns = df.columns)
scaled_data_df

Out[363]:
   Murder  Assault  UrbanPop  Rape
0  1.255179  0.790787 -0.526195 -0.003451
1  0.513019  1.118060 -1.224067  2.509424
2  0.072361  1.493817  1.009122  1.053466
3  0.234708  0.233212 -1.084492 -0.186794
4  0.281093  1.275635  1.776781  2.088814
5  0.025976  0.402909  0.869548  1.883901
```

Now, if the variables have been standardized as we requested, each variable should now have a mean of 0 and variance of 1. Let's confirm this by obtaining first the means, using once more `np.mean()`:

```
np.mean(scaled_data_df)

Out[578]:
Murder      -8.437695e-17
Assault      1.298961e-16
UrbanPop    -4.263256e-16
Rape         8.326673e-16
dtype: float64
```

We can see that within rounding error, the mean of each variable is now equal to 0 (recall that `-8.437695e-17` means to move the decimal point 17 positions to the left, hence making it an extremely small number). Since the variables have been standardized, the variance of each variable should equal 1, as we can easily confirm via `np.var()` of our scaled data:

```
np.var(scaled_data_df)

Out[579]:
Murder      1.0
Assault      1.0
UrbanPop     1.0
Rape         1.0
dtype: float64
```

Based on the above then, we have confirmation that Python has indeed correctly standardized the data. Let's proceed now to run the PCA on the scaled data. We will request a total of four components, which is the maximum number of components possible for this data since there are only four variables. We specify the number of components via the statement `n_components=4`:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=4)
pca.fit(scaled_data_df)

Out[364]:
PCA(copy=True, iterated_power='auto', n_components=4, random_
state=None,
      svd_solver='auto', tol=0.0, whiten=False)

pca.components_
Out[366]:
array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
       [ 0.41818087,  0.1879856 , -0.87280619, -0.16731864],
       [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
       [ 0.6492278 , -0.74340748,  0.13387773,  0.08902432]])
```

In the above, **each component is a row, not a column**. For example, the first component has loadings of 0.53589947, 0.58318363, 0.27819087, and 0.54343209, and so on for the remaining components. The first component is thus

$$0.53(\text{Murder}) + 0.58(\text{Assault}) + 0.27(\text{UrbanPop}) + 0.54(\text{Rape})$$

The second component is

$$0.41(\text{Murder}) + 0.18(\text{Assault}) - 0.87(\text{UrbanPop}) - 0.16(\text{Rape})$$

and so on for the last two components. So, what does each component represent? Notice that the first component seems “composed of” primarily murder, assault, and rape. The coefficient for UrbanPop is a bit less with a loading of only 0.27, hence maybe the first component represents a combination of crimes. The second component is a bit more heavily weighted on UrbanPop (though the weight for murder is also creeping up a bit too). This component is much more difficult to make sense of. And this fact is a good thing, at least for demonstration and pedagogical purposes. It exemplifies (again) that “naming components” can be very elusive. While components analysis has done its job, it does not mean the resulting components will “make sense” on a substantive level. They make sense **mathematically**, yes, but they may not make sense **substantively**. This last point cannot be emphasized enough if you are to have a clear-headed understanding of what components analysis accomplishes vs. what it does not.

We can easily obtain the proportion of variance explained using `pca.explained_variance_ratio`:

```
print(pca.explained_variance_ratio_)
[0.62006039 0.24744129 0.0891408 0.04335752]
```

Note that the sum of the proportions equals 1.0. The sum of the variances for each component will also sum to 4. This is as a result of performing the PCA on the **standardized data** rather than the **raw data**. Had we performed it on the raw data, then the sum of variances should have equaled the **sum of the variances of the raw variables** inputted into the analysis. Let us verify this. We perform the PCA on the raw data:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=4)
pca.fit(df)

Out[368]:
PCA(copy=True, iterated_power='auto', n_components=4, random_
state=None,
      svd_solver='auto', tol=0.0, whiten=False)

pca.components_

Out[369]:
array([[ 0.04170432,  0.99522128,  0.04633575,  0.0751555 ],
       [ 0.04482166,  0.05876003, -0.97685748, -0.20071807],
       [ 0.07989066, -0.06756974, -0.20054629,  0.97408059],
       [ 0.99492173, -0.0389383,  0.05816914, -0.07232502]])
```

Next, we compute the actual variances of each component:

```
print(pca.explained_variance_)
[7.01111485e+03 2.01992366e+02 4.21126508e+01 6.16424618e+00]
```

We then sum these variances:

$$7,011.11 + 201.99 + 42.11 + 6.16 = 7,261.37$$

The total variance of the components is thus equal to 7,261.37. If we sum the variances we computed earlier using `np.var()`, that sum will equal 7,116.15. However, if PCA is working correctly, the sum of variances of the original variables should be equal to that of the components. But 7,261.38 is clearly different from 7,116.15. This is where the denominator issue discussed earlier matters. When we use pandas' `df.var()`, we obtain instead:

```
df.var()
Out[395]:
Murder      18.970465
Assault     6945.165714
UrbanPop    209.518776
Rape        87.729159
dtype: float64
```

The sum of these variances is equal to 7,261.38, which agrees with the sum of the variances of the components just above. Again, this is different from the variance computed by numpy, because pandas is using the “ $n - 1$ ” denominator by default, whereas `np.var()` used only n in the denominator (unless we specify `np.var(df, ddof=1)`). We can also quite easily obtain the standard deviations of the original variables, as they are merely the square roots of the above variances:

```
df.std()
Out[404]:
Murder      4.355510
Assault     83.337661
UrbanPop    14.474763
Rape        9.366385
dtype: float64
```

Summing these squared standard deviations is also equal to the sum of the variances for the components (verify this for yourself).

10.9 Plotting the Components

A convenient way to obtain plots of the components solution is to use the **pca** library in Python or the **yellowbrick** package. We use both below. Because these packages do not come already installed with the Anaconda distribution, we will need to install them via **PIP** (refer to our previous discussion in Chapter 2 on how to do this in Anaconda):

```
pip install pca
from pca import pca
```

```
pip install yellowbrick
import yellowbrick
```

We then, as before, run the model obtaining all four components:

```
model = pca(n_components=4)
```

We will now first generate plots on the standardized data. To generate these plots, we use the scaled data with no headers (i.e. recall that we had put the headers back on earlier; do not use that dataframe, use the one with no headers instead):

```
X = scaled_data
results = model.fit_transform(X)
```

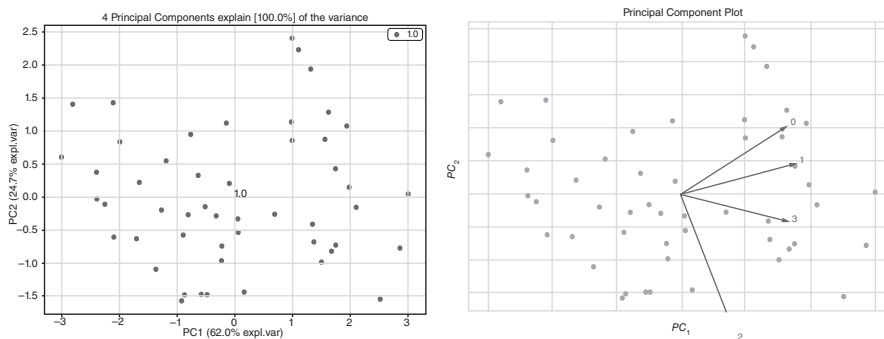
Requesting the results object will provide much information (e.g. to obtain outlier details, try `print(results['outliers'])`, most of which we do not show here since our objective is simply to produce a plot (for details, see the PCA library). We do, however, print the loadings that result:

```
results
Out[305]:
{'loadings': 1          2          3          4
PC1  0.535899  0.583184  0.278191  0.543432
PC2  0.418181  0.187986 -0.872806 -0.167319
PC3 -0.341233 -0.268148 -0.378016  0.817778
PC4  0.649228 -0.743407  0.133878  0.089024
```

We first produce a generic plot before generating a biplot. The first plot is a scatter-plot of the first two component scores (left) using the `pca` package, while the second plot (right) is a biplot using `yellowbrick`:

```
fig, ax = model.scatter()
```

```
fig, ax = model.biplot(n_feat=4)
```

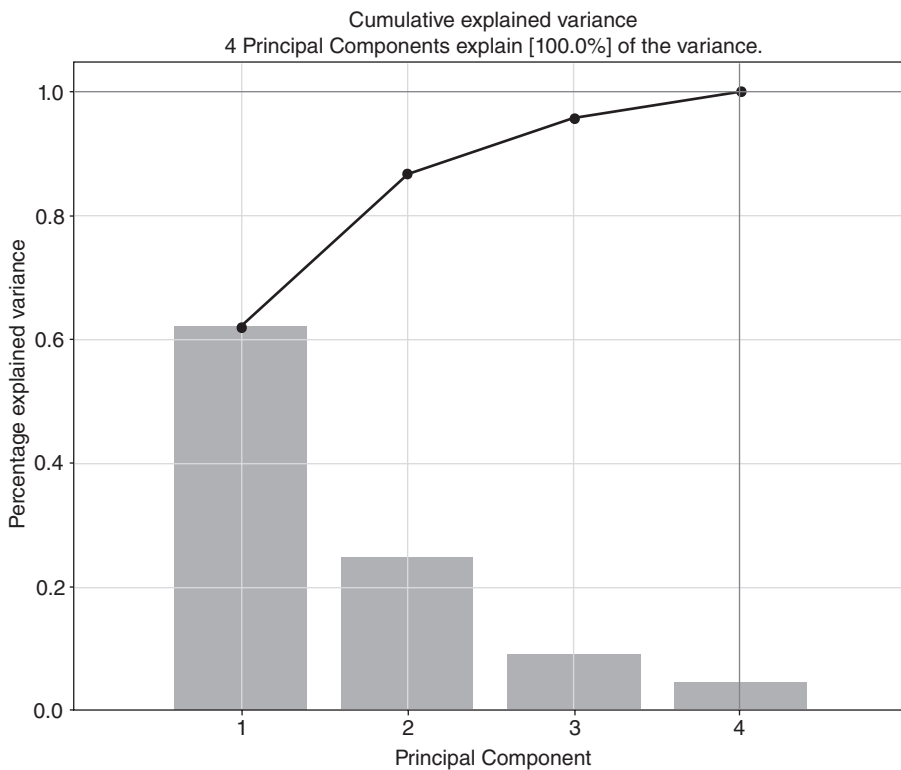


In the biplot (right), we can see that the first PC, scaled along the x-axis, has variables 0, 1, and 3 (corresponding to murder, assault, and rape, respectively) that are further to the right of the plot than the third variable ("2" in the plot, since the variables are

numbered 0, 1, 2, 3), corresponding to urbanpop. In agreement with the component loadings in the above table, these variables are most “important” on the first component. Now, consider the plot from the vantage point of the second component (PC2). From this vantage point, we can see that the third variable (“2” in the plot, urbanpop) dominates the component, while the other three variables are much less in magnitude. Again, this agrees with what we are seeing in the loadings output. Hence, the biplot allows us to visualize variables across components in terms of their sizes on each component.

The following plots the variances explained by each of the components. As expected, the first component accounts for the majority of the variance in the variables, while the fourth and last component accounts for the least. The sum total of components extracted account for 100% of the explained variance. This is not surprising, since, as mentioned, PCA maintains the total variance in the original variables. It simply (and perhaps rather anticlimactically) re-expresses it into orthogonal dimensions. Thus, if the total variance did not sum to 100%, it would be suggestive of a problem with the components analysis:

```
fig, ax = model.plot()
```



We can also generate a **3D plot** (not shown), which, as a result of the new dimension, we can plot the first three components rather than the first two (`fig, ax = model.scatter3d()`) as well as the corresponding biplot (`fig, ax = model.biplot3d(n_feat=4)`).

Review Exercises

1. Discuss the purpose of the **PCA**. What might motivate a researcher to consider such a method for his or her variables?
2. Why is it incorrect to say that PCA “**uncovers underlying components**” unless one actually knows what this statement means? What is the danger of interpretation in not understanding the technical base of PCA yet still using this phraseology?
3. Your colleague says to you, “PCA issues a **transformation** on data and nothing more.” Do you agree with your colleague? Why or why not?
4. Why is it true that the **total amount of variance** in the variables before conducting a PCA is the same found in the components after conducting one? Explain.
5. Critically evaluate the use of **icebergs** as an analogy to what PCA accomplishes. Why might such an analogy be problematic?
6. Discuss PCA in terms of an **eigenvalue–eigenvector** problem. What is actually taking place when computing these mathematical entities? Describe the process.
7. Is running a PCA on a **covariance matrix** the same as that of running one on a **correlation matrix**? Why or why not?
8. Consider the **Breast Cancer Wisconsin (Original)** data set on several attributes related to breast cancer, including clump thickness, uniformity of cell size, up to and including mitoses. Consider all 10 of these variables, all measured on a continuous scale. Perform a PCA on this data to determine whether fewer than 10 dimensions can account for the majority of the variance in the original variables, and, if so, attempt to interpret these components. What proportion of the total variance do these components account for?
9. Confirm for your analysis in exercise 8 that the total variance of all components equates to the total variance of the original variables. Obtain a plot that summarizes the results of the PCA.
10. Consider the idea of extracting components from a digital image. Suppose that underlying a digital image made up of 100 variables are five components. How might this use of PCA differ from using it to extract components from social variables? Is it different at all? Explain.